TechSolve IT — Support Performance Dashboard

1. Where did your datasets come from, and why did you choose them?

The core dataset was the ticket export provided for this exercise. It's a single table covering customer, account, ticket, resolution and satisfaction fields: ticket_id, customer/company details, category, priority, status, timestamps, resolution and SLA fields, CSAT, channel, and technical metadata such as browser/OS.

For the external data source requirement, I chose NZ Public Holidays. I selected this over alternative such as NZ weather data for a few reasons:

- It joins cleanly to the ticket data on ticket_created_date, with no ambiguity or need for approximate matching.
- It directly tests an operationally relevant hypothesis for a support business. Tickets were logged on or near public holidays take longer to resolve because of reduced staffing.
- It's a clean, authoritative, low-noise reference dataset, which kept the join and blending logic simple and let me focus effort on the ticket data itself.

Assumption: I treated the public holiday field as a national-level flag rather than region-specific NZ holidays.

2. Describe your process for working through this scenario.

Part 1: Source, Combine & Prepare

- Imported the ticket export and the NZ public holiday data into Power Query.
- Replaced blanks with meaningful defaults, shown below. For company_name specifically, I confirmed the blanks by cross-checking against the account_type column before deciding "Residential Customer" was the right default, rather than assuming.

Column	Blank replaced with
company_name	Residential Customer (confirmed via account_type)
billing_contact_email	N/A
account_manager, team, assigned_to	Unassigned
browser	Unknown
csat_score	Left as Null, not 0 (0 would misrepresent "no rating given" as "worst rating given")

- Standardised the category field using TRIM, Clean and Capitalise Each Word to fix spacing, hidden characters and inconsistent casing, then built a Text.Contains based mapping to consolidate near duplicate labels (for example "Acct Suspension" and "Account Suspension", or "Bug" and "Bug Report", or "Refund Req" and "Refund Request") into 10 clean categories, while preserving the original raw text as a sub-category for transparency.
- Removed customer_email and billing_contact_email as PII not required for reporting, while retaining issue_description and resolution_notes as free text for the Part 3 agent to reason over.
- Identified and corrected invalid dates: 9 tickets had ticket_created_date values of 1970 or 2099, clear data entry or system errors, which were nulled via a new valid_ticket_date column bounded to 2022 to 2026.
- Found that some Open, Pending and In Progress tickets had populated resolution dates and resolution_time_hours despite not being finished, so I created a new valid_ticket_resolved_date column that only keeps the resolved date when status is Closed or Resolved.
- Joined the NZ Public Holidays table to Tickets on date, and later cleaned the resulting Is_Public_Holiday field, which had only returned blank or "yes", into an explicit "Yes"/"No" column for cleaner charting.

Part 2: Visualise

- Built measures for Avg Resolution Hours, Open Tickets, Avg First Response and Resolved On Time. After finding the supplied sla_breached field gave results inconsistent with the actual resolution-time-versus-target maths, I also built a corrected SLA Breach / SLA Breach Rate measure comparing resolution_time_hours directly against sla_target_hours, restricted to resolved tickets.
- Built six report pages: Overview (KPIs, category/status/SLA by priority), Performance Analysis (resolution time by category and priority, team performance, public holiday impact), Category & Sub-Category Breakdown (matrix drill-down plus a data-quality callout on label consolidation), Ticket Insights (top accounts, region, industry, channel, account manager, service area), Account Value vs Service Performance (contract value against response, resolution and SLA metrics), and Team Status (ticket status by team and individual agent, plus response and resolution time by category per team).
- The Team Status page surfaced the one genuinely non-flat finding in the whole analysis: ticket volume is heavily skewed by team, with Support handling 58,303 tickets, more than three times the next largest team (Billing, at 17,913). SLA breach and escalation rates still stay flat, around 50 percent, across all teams, which suggests Support may be understaffed relative to its workload even though the breach rate itself doesn't spike there. This looks like a staffing or capacity question rather than a team-performance one.

Part 3: AI Agent

- Built a natural-language agent using VS Code, GitHub Copilot in Agent mode, and the local Power BI Modeling MCP Server, connected directly to the live semantic model behind the dashboard, and systematically tested it against numbers I had already validated by hand.
- Recorded a screen capture (screen_recording.mp4, attached) walking through the finished dashboard and a live Copilot Chat Q&A session.

3. What tools and programs did you use, and why those over alternatives?

Tool	Used for	Why chosen over alternatives
Power BI Desktop	Data modelling, DAX measures, dashboard build	Free, industry standard, and the brief explicitly named it ("Power BI or equivalent")
Power Query	Cleaning, consolidation, joins	Built into Power BI, with an auditable step-by-step transformation log rather than opaque scripting
NZ Public Holidays dataset	External data source	Clean join key (date), a directly testable business hypothesis, and low noise
VS Code + GitHub Copilot (Agent mode)	Part 3 AI agent	Free tier available, and Agent mode can call external tools rather than only generating text
Power BI Modeling MCP Server (local)	Connects the agent to the live semantic model	Runs locally against the open PBIX with no Fabric/Premium workspace or tenant licence required, which mattered once I found I didn't have workspace access for Copilot Studio's cloud option

I originally planned to use Microsoft Copilot Studio connected to a published Power BI dataset. I don't have access to a Fabric workspace, so I used the local MCP approach above instead.

4. What would you do differently? What did you find most challenging?

What I'd do differently

- Validate the SLA breach logic against the raw resolution-time-versus-target figures earlier. I initially trusted the supplied sla_breached field and a naive AVERAGE for resolution time, and only caught both issues after cross-checking the maths, which cost time later in the process.
- I spent a lot of time troubleshooting why the AI agent kept returning wrong numbers, including restarting the connection and reconnecting the MCP server more than once, before realising I should have built a checklist of verified numbers from Power BI Desktop and the raw Excel export upfront, and checked every agent response against it from the start.

- Spend less time re-testing dimensions that were already clearly flat (category, team, priority, subscription type, service area, customer segment) once the pattern was established and move sooner to writing up the conclusion that follows from that pattern that SLA breaches are not caused by any single category, team, priority level or customer type, but are systemic across the whole ticket process.

Most challenging

- Diagnosing why the MCP agent's answers were inconsistent. The same DAX formula returned different results in Power BI Desktop versus Copilot Chat for several measures (Total Tickets, Avg CSAT, Escalation Rate, Avg First Response), and in at least one case the agent explained what a DAX query would do in words rather than actually executing it. Distinguishing "the agent picked the wrong field" (fixable by supplying business logic) from "the agent didn't actually query the model" (not fixable by better prompting) took a lot of trial and error.
- The dataset itself being very evenly distributed across almost every dimension (category, team, priority, channel, region-normalised rates, subscription tier, complexity score) made it hard to find a genuine differentiator. Most cuts of the data returned effectively flat results, which is a legitimate finding but required many iterations to confirm it wasn't a modelling mistake on my part each time.

5. Roughly how long did you spend on each phase?

Phase	Approx. time
Part 1: Data collection, Source, combine, clean and prepare	8 - 9 hours
Part 2: Dashboard build (6 pages, measures, iteration)	7 - 8 hours
Part 2: Validation and root-cause investigation	5 - 6 hours
Part 3: AI agent setup (Copilot Studio attempt, pivot to local MCP)	2 - 3 hours
Part 3: Agent testing and accuracy verification against ground truth	7 - 8 hours
Written review and documentation	6 hours

Total: roughly 35 to 40 hours across the full exercise.

6. Did you use AI tools during this practical? If so, how?

Yes, in two distinct ways.

As the Part 3 deliverable itself

VS Code, GitHub Copilot in Agent mode, and a local Power BI Modeling MCP Server, connected to the live semantic model, formed the natural-language Q&A agent requested by the brief.

As a working aid throughout Parts 1 and 2

I used an AI assistant (Claude, ChatGPT) conversationally while building, to check DAX logic and talk through what a given result might mean. I treated its suggestions as a starting point, not a source of truth. Every DAX formula and finding it proposed was tested in Power BI Desktop and cross-checked against known values, such as dashboard totals and the raw Excel export, before being kept.

Key finding from testing the Part 3 agent

This testing produced one of the more interesting outcomes of the whole exercise. The MCP-connected agent's reliability was inconsistent even within a single session. It correctly evaluated some measures (Avg Resolution Hour: 120.72, matching Power BI exactly) but returned incorrect values for others (Total Tickets: 100 instead of 100,851; Avg CSAT: 4.2 instead of 3.0; Avg First Response: 12.5 instead of 36.30), despite the underlying DAX being verified identical across the raw Excel export, Power BI Desktop, and the model.

In one instance, the agent appeared to explain what a DAX query would return in words rather than actually executing it against the model. The agent also generated a plausible-sounding but unsupported causal narrative, claiming Urgent tickets breached SLA at 98.57 percent and that breaches were "concentrated" in specific teams, which directly contradicted my own validated dashboard analysis (breach rate is flat across priority and team, roughly 85 to 93 percent throughout).

Every AI-generated number and claim used in the final dashboard and this review was cross-verified against Power BI Desktop and/or the raw data before being trusted. None was taken at face value.

A screen recording (screen_recording.mp4) is attached alongside this review and the GitHub repository, showing a live Copilot Chat session as direct supporting evidence for the findings described above.

Assumptions

- Public holidays were treated as a single national NZ calendar rather than region-specific observances, given the join constraints described in Q1.
- Where csat_score was blank, this was treated as "no rating given" and excluded from averages (via Null), not treated as a 0 or worst-possible score.
- Tickets with a status other than Closed or Resolved were treated as not yet having a valid resolution time, and were excluded from resolution-time and SLA-breach calculations rather than counted as compliant or non-compliant by default.
- Given the unusually even distribution across almost all dimensions of the ticket data (category, team, priority, subscription tier, channel, service area), I assumed this reflects a synthetically generated dataset rather than genuine operational variance, and stated this explicitly as a limitation rather than forcing an artificial narrative onto flat results.

Project Repository

GitHub link: https://github.com/RamuLincoln/techsolveIT